

Introduction

Every Python developer writes `for x in my_list:` before they've ever heard the word "iterator." That's by design — Python hides a whole protocol behind that syntax so beginners don't have to think about it. But the moment you're handling large datasets, streaming files, or building a data pipeline, that hidden machinery stops being optional trivia and starts being the difference between code that runs and code that crashes your machine.

This also happens to be one of the most common areas interviewers probe when they want to know if a candidate actually understands Python, versus just being able to use it. "What's the difference between an iterable and an iterator?" and "How would you handle a dataset too large to fit in memory?" are staple questions, and generators are almost always the expected answer to the second one.

Understanding this topic isn't about memorizing `__iter__` and `__next__`. It's about understanding *why* Python was designed to compute values lazily in the first place, and knowing when that laziness helps you and when it's just unnecessary indirection.

Problem Overview

Here's the core question we're answering: if you ask Python for a huge sequence of numbers, why does one approach run instantly with almost no memory footprint, while a nearly identical-looking approach can freeze or crash your machine?

Concretely:

```
range(1_000_000_000)      # instant, negligible memory
list(range(1_000_000_000)) # can exhaust memory on a typical laptop
```

Both describe "a billion numbers." Only one of them actually creates a billion numbers right now. The rest of this article is about understanding that distinction from the ground up — starting with what a `for` loop is actually doing when you're not looking.

Example

Let's make the abstract concrete before touching any theory.

```
numbers = [10, 20, 30]

for n in numbers:
    print(n)
```

Output:

```
10
20
30
```

Nothing surprising here — but this innocent loop is secretly doing three distinct things: asking the list for an iterator, repeatedly asking that iterator for the next value, and catching a signal that tells it when to stop. We'll expose each of those steps manually next.

Intuition

Think about how an experienced engineer would explain a `for` loop to a machine that has no shorthand for "loop over this."

They wouldn't say "go through the list." They'd say something closer to: "Get me something that can hand out values from this list one at a time. Ask it for a value. Do something with it. Ask again. Keep asking until it tells you there's nothing left."

That's it. That's the entire mental model. A `for` loop is not a primitive operation — it's syntactic sugar over a two-step handshake:

1. Call `iter()` on the thing you're looping over. This returns an **iterator** — an object that knows how to produce values one at a time and remembers where it left off.
2. Call `next()` on that iterator repeatedly. Each call either returns the next value, or raises `StopIteration` when there's nothing left, which the `for` loop silently catches to end the loop.

The key distinction that trips people up: a list is **iterable** (it knows how to produce an iterator via `__iter__`), but a list is not itself the iterator doing the work. The iterator is a separate object with its own memory of position.

A good analogy: a list is the deli counter's number roll. The iterator is the ticket dispenser — it doesn't hold every number at once, it just remembers which number it handed out last and produces the next one on demand.

Once you see the loop this way, the memory question answers itself. If producing "the next value" doesn't require having *all* the values already computed, you never need to store them all. That's the entire insight behind `range()`, generators, and every lazy-evaluation trick in Python.

Brute Force Solution

The "brute force" here isn't an algorithm in the traditional sense — it's the naive approach to representing a large sequence: build the whole thing up front.

Idea: Compute every value in the sequence immediately and store it in a list.

```
data = list(range(1_000_000_000))
```

Algorithm: 1. Allocate memory for a list. 2. Compute each value in the range. 3. Store all of them before returning control to the caller.

Advantages: - Simple to reason about — the data is just *there*. - Supports random access (`data[500_000]`) and repeated iteration for free. - Works fine for small-to-medium sequences.

Disadvantages: - Every value must exist in memory simultaneously, even if you only ever touch each one once. - Doesn't scale to huge or infinite sequences. - Wastes time computing values you may never use.

Complexity: - Time: $O(n)$ to build the list. - Space: $O(n)$ — this is the actual problem. A billion integers, even at Python's per-object overhead, is enough to exhaust RAM on a typical machine.

Optimal Solution

The optimal approach is to never materialize the full sequence — only compute a value the instant it's asked for, and immediately forget it afterward. This is what `range()` does natively, and it's the same principle behind hand-written iterators and generators.

There are two ways to build this behavior yourself:

1. A hand-built iterator, implementing the protocol explicitly:

```
class CountUp:
    def __init__(self, start, end):
        self.current = start
        self.end = end
```

```
def __iter__(self):
    return self

def __next__(self):
    if self.current >= self.end:
        raise StopIteration
    value = self.current
    self.current += 1
    return value
```

2. A generator function, using `yield` to get the same behavior with far less code:

```
def count_up(start, end):
    current = start
    while current < end:
        yield current
        current += 1
```

Calling `count_up(0, 3)` doesn't run any code — it returns a generator object, paused before the first line, like a video paused at frame zero. Only when you call `next()` does execution actually proceed, running until it hits `yield`, handing back a value, and freezing exactly there. The next `next()` call resumes execution right after that `yield`, not from the top of the function.

Step	What happens
<code>gen = count_up(0, 3)</code>	Generator object created, nothing executed yet
<code>next(gen)</code>	Runs to first <code>yield</code> , returns <code>0</code> , pauses
<code>next(gen)</code>	Resumes after <code>yield</code> , returns <code>1</code> , pauses
<code>next(gen)</code>	Resumes, returns <code>2</code> , pauses
<code>next(gen)</code>	Loop condition fails, raises <code>StopIteration</code>

Because nothing is computed until asked for, and nothing is retained after being yielded, memory use stays constant regardless of how large — or infinite — the sequence is.

Python Code

```
def count_up(start: int, end: int):
    """Yield integers from start up to (but not including) end, one at a time."""
    current = start
    while current < end:
        yield current
        current += 1

def evens_generator_expression(limit: int):
    # Generator expression: computes nothing until iterated
```

```

return (n for n in range(limit) if n % 2 == 0)

if __name__ == "__main__":
    gen = count_up(0, 5)
    print(list(gen)) # [0, 1, 2, 3, 4]

    evens = evens_generator_expression(10)
    print(list(evens)) # [0, 2, 4, 6, 8]

```

Code Walkthrough

- `def count_up(start, end):` — because the function body contains `yield`, calling this returns a generator object, not a value. No code inside runs yet.
- `current = start` — this line only executes the first time `next()` is called on the generator.
- `while current < end:` — the loop condition is re-checked every time the generator resumes.
- `yield current` — this is the pause point. Execution stops here and control returns to the caller, but every local variable (`current`, in this case) stays alive in memory for when it resumes.
- `current += 1` — runs *after* the generator is resumed with another `next()` call, not immediately after the `yield`.
- `evens_generator_expression` — the round-bracket version of a list comprehension. `(n for n in range(limit) if n % 2 == 0)` builds nothing; it's a promise to compute values on demand, identical in spirit to the generator function above but expressed in one line.

Dry Run

Trace `count_up(0, 3)` step by step:

1. `gen = count_up(0, 3)` → generator object created, paused before `current = start`.
2. `next(gen)` → runs `current = 0`, checks `0 < 3` (true), yields `0`, pauses.
3. `next(gen)` → resumes after yield, runs `current += 1` → `current = 1`, checks `1 < 3` (true), yields `1`, pauses.
4. `next(gen)` → resumes, `current = 2`, checks `2 < 3` (true), yields `2`, pauses.
5. `next(gen)` → resumes, `current = 3`, checks `3 < 3` (false), loop exits, function returns, `StopIteration` raised.

At no point does the generator hold more than one value in memory — `current` is the only state it carries.

Complexity Analysis

List-based approach: - Time: $O(n)$ to build. - Space: $O(n)$ — every value exists simultaneously in memory.

Generator-based approach: - Time: $O(n)$ total if you consume every value — same total work, just spread out over time. - Space: **$O(1)$** — only the current position and a handful of local variables are kept, regardless of how many values the sequence contains.

This is why `range(1_000_000_000)` is instant: `range` objects don't generate values until indexed or iterated, and they store only `start`, `stop`, and `step` — three integers, not a billion.

Alternative Solutions

`itertools` module: For more advanced lazy sequences (infinite counting, chaining, filtering, batching), `itertools.count()`, `itertools.islice()`, and friends give you the same $O(1)$ -memory behavior as hand-written generators, with a well-tested C implementation underneath. If you're reaching for a custom generator that just counts, filters, or chains, check `itertools` first — it's almost always faster and more concise.

Plain lists: Still the right choice when the data is small, needs random access, or must be iterated more than once. Generators are single-use — once exhausted, they're done, and there's no rewinding. Don't reach for laziness by default; reach for it when the size, streaming nature, or single-use pattern of the data actually calls for it.

Edge Cases

- **Empty range:** `count_up(5, 5)` — the `while` condition fails immediately, the generator yields nothing, and the very first `next()` call raises `StopIteration`.
- **Reversed bounds:** `count_up(5, 0)` behaves the same as empty — `5 < 0` is false, so nothing is yielded. If descending order is intended, the step logic needs to be written explicitly; it won't happen automatically.
- **Exhausted generator reused:** Calling `next()` again after `StopIteration` has already been raised once will raise it again — a generator cannot be reset or rewound. You need a fresh call to the generator function.
- **Iterating a generator twice:** `for x in gen: ...` followed by a second `for x in gen: ...` on the *same* object will produce nothing the second time, since the generator is already exhausted. This surprises people coming from list-based code.

- **Very large but not infinite range with `range()` :** `range()` handles this natively and efficiently — it's the built-in demonstration of $O(1)$ memory over an arbitrarily large sequence.

Common Mistakes

1. **Confusing iterable and iterator.** Assuming a list itself has state like "current position" — it doesn't; the iterator returned by `iter(list)` does.
2. **Trying to `len()` a generator.** Generators don't know their total size in advance in the general case, so `len()` raises `TypeError`.
3. **Reusing an exhausted generator.** Expecting a second loop over the same generator object to work, when it's already been fully consumed.
4. **Wrapping small, already-materialized data in a generator "for efficiency."** This adds complexity with zero memory benefit if the data is small and reused.
5. **Forgetting that local state persists across `yield`.** Assuming variables reset between `next()` calls, when in fact they're preserved exactly as they were when the generator paused.
6. **Using a list comprehension when a generator expression would do,** inside something like `sum(...)` or `any(...)`, where the full list is never needed — `sum([x for x in range(n)])` builds an unnecessary list; `sum(x for x in range(n))` doesn't.

Interview Questions

- What's the difference between an iterable and an iterator?
- What exception ends a `for` loop internally, and why don't you see it?
- How would you implement a custom iterator class from scratch?
- What's the difference between `return` and `yield` in a function?
- When would you choose a generator over a list, and when would a list still be better?
- Can a generator be iterated more than once? Why or why not?
- How does `range()` achieve constant memory usage for huge ranges?

Similar Problems

- **LeetCode 284 (Peeking Iterator)** — directly tests understanding of the iterator protocol and how to look ahead without breaking `next()`'s state.
- **LeetCode 173 (Binary Search Tree Iterator)** — requires implementing `__next__` and `hasNext` with $O(1)$ amortized time, a real-world use of custom iterators.

- **LeetCode 341 (Flatten Nested List Iterator)** — combines lazy evaluation with recursive structures, a natural extension of generator thinking.
- **LeetCode 251 (Flatten 2D Vector)** — another custom iterator implementation exercise, good for practicing `__iter__` / `__next__` by hand.

Key Takeaways

- Every `for` loop is `iter()` followed by repeated `next()` calls, ending on a caught `StopIteration`.
- An iterable produces an iterator; the iterator is the thing that actually remembers position and produces values.
- `yield` turns a function into a generator: it pauses at each `yield` and resumes exactly there on the next call, preserving all local state in between.
- Generator expressions give you the same laziness as generator functions, in list-comprehension syntax with parentheses instead of brackets.
- Lists compute and store everything up front; generators compute one value at a time and remember none of the past ones — that's the entire memory story.
- Use generators for huge, streaming, or single-pass data. Use lists when the data is small or needs to be iterated more than once.

Watch the Video

This entire walkthrough is demonstrated live, including hand-tracing a generator pausing and resuming step by step — watch it here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around taking the tools developers use every day — loops, data structures, language features — and opening them up to see exactly how they work underneath. The goal isn't just to show *what* to write, but to build the intuition for *why* it works, so the next unfamiliar problem feels less unfamiliar.

Call To Action

If this made the iterator protocol click, leave a like on the video — it's the easiest way to tell me the pacing and depth are working. Subscribe on YouTube for the rest of the series, and subscribe to the Substack for the written breakdowns that go even deeper than the videos. Drop a comment with what

you'd like explained next, and share this with anyone prepping for interviews or just trying to actually understand Python instead of memorizing syntax.

Quick Review

Generator with 1B numbers doesn't crash memory—why?

Yields one value at a time instead of building the full list; $O(1)$ memory vs $O(n)$ for a list.

What does 'for x in obj' secretly call?

`iter(obj)` to get an iterator, then repeated `next()` until `StopIteration` is raised.

Iterable vs Iterator: what's the difference?

Iterable has `__iter__` (can produce an iterator); Iterator has `__next__` and `__iter__`, tracks its own position, exhausts after one pass.

Common bug: reusing a generator/iterator a second time?

It's already exhausted—empty results, no error. Iterators are single-use; recreate the generator to iterate again.

yield vs return in a function?

`return` ends the function once; `yield` pauses execution, saves state, and resumes from that point on the next `next()` call.

Generator expression vs list comprehension?

Generator expr `(x for x in y)` is lazy, $O(1)$ memory; list comp `[x for x in y]` builds the whole list eagerly, $O(n)$ memory.